

[INSTITUTE / COLLEGE NAME]

[Department Name]

A MINI PROJECT REPORT ON

AURIX

GLOBAL AUTOMOTIVE INTELLIGENCE

A Premium 3D Interactive Automotive Encyclopedia, Visualizer & Configurator Platform

Submitted By

S.No.	Name of Student	Roll No. / Enrollment No.
1.	[Student Name]	[Roll No.]
2.	[Student Name]	[Roll No.]
3.	[Student Name]	[Roll No.]
4.	[Student Name]	[Roll No.]

Under the Guidance of

[Guide / Faculty Name]

Academic Year: [20____ - 20____]

TABLE OF CONTENTS

1. Name of Students.....	
2. Problem Statement.....	
3. Project Description.....	
4. Software Used.....	
5. Programming Language Used.....	
6. Application Area.....	
7. Flowchart.....	
8. Input.....	
9. Screenshot of Output.....	
10. Conclusion.....	

References

Note: Page numbers may be filled in by hand or updated automatically once the document is finalized in Microsoft Word.

1. NAME OF STUDENTS

This mini project, titled "AURIX — Global Automotive Intelligence", has been designed and developed as a group project by the following students, in partial fulfilment of the requirements for the award of the degree/diploma in the respective course.

S.No.	Name of Student	Roll No. / Enrollment No.	Class / Semester
1.	[Student Name 1]	[Roll No.]	[Class / Sem]
2.	[Student Name 2]	[Roll No.]	[Class / Sem]
3.	[Student Name 3]	[Roll No.]	[Class / Sem]
4.	[Student Name 4]	[Roll No.]	[Class / Sem]

Guide / Mentor Details

Guide Name	Designation	Department
[Guide Name]	[Designation]	[Department]

Note: Names, roll numbers and guide details are placeholders and should be replaced with the actual student information before final submission and printing.

2. PROBLEM STATEMENT

Automotive information available to enthusiasts, students, and prospective buyers today is scattered across dozens of disconnected sources — manufacturer brochures, third-party review sites, forums, video reviews, and static specification tables. Each of these sources presents data in an inconsistent format, rarely links related information (such as how a country's automotive industry shaped a manufacturer, or how a manufacturer's model lineage evolved into distinct variants), and almost never allows a user to visually and interactively explore a vehicle's mechanical construction.

Furthermore, while several premium automotive brands offer 3D configurators on their own websites, these tools are restricted to a single manufacturer's product line, are commercially oriented toward sales rather than education, and do not allow a user to compare vehicles across brands, countries, or powertrain types. There is no unified, neutral, and technically rich platform that combines:

- Global, country-wise organisation of automotive manufacturers and their history.
- A relational database connecting countries, manufacturers, models, variants, and individual mechanical parts.
- Interactive 3D visualization of vehicles, including an exploded component view and an engineering-mode technical visualization.
- Structured specification data (performance, engine, transmission, dimensions, fuel/energy) presented consistently across every vehicle regardless of powertrain type.
- Advanced search, filtering, and side-by-side comparison of multiple vehicles.

This creates a genuine gap between the way automotive information is currently presented on the web and the way a modern, data-driven, visually engaging platform could present it. Students, automotive enthusiasts, and researchers are forced to manually collect and cross-reference information from multiple unreliable or commercially biased sources, which is time-consuming and often inconsistent.

Statement of the Problem

"To design and develop a unified, database-driven, 3D-interactive web platform that presents global automotive information — spanning countries, manufacturers, models, variants, specifications, and individual vehicle parts — in a structured, comparable, and visually engaging manner, while remaining technically extensible for future data growth and administrative management."

3. PROJECT DESCRIPTION

3.1 Overview of the Project

AURIX — Global Automotive Intelligence is a full-stack, database-backed web application that functions simultaneously as a 3D automotive configurator, a global automotive encyclopedia, an engineering visualization platform, and a car comparison tool. It is built using Next.js and React on the frontend, React Three Fiber / Three.js for real-time 3D rendering, Tailwind CSS for a premium dark-luxury visual language, and Supabase (PostgreSQL) as the persistent, relational backend.

The platform organises automotive data along a clear hierarchy — Country → Manufacturer → Model → Variant → Specification → Parts — allowing a user to start from a broad entry point (e.g., a country or a manufacturer) and progressively drill down into precise mechanical detail (e.g., the exact torque figure of a single engine variant, or the material composition of a turbocharger).

3.2 Objectives

- To design a normalized relational database schema capable of representing countries, manufacturers, models, variants, specifications, and parts without data duplication.
- To build an interactive 3D vehicle viewer supporting orbit, zoom, pan, and preset camera views (front, rear, side, interior, engine, wheels, brakes).
- To implement an exploded-view mode that visually separates a vehicle into its major mechanical assemblies with an accompanying information panel.
- To provide a country-wise and manufacturer-wise browsing experience supported by a global search and multi-parameter filtering system.
- To implement a side-by-side comparison tool for two to four vehicles across performance, engine, dimensions, and other categories.
- To ensure the platform is responsive, accessible, and performant across desktop and mobile devices.
- To structure the application so that an administrative content-management layer can be added cleanly in the future, without re-architecting the system.

3.3 Scope of the Project

The current scope of the project covers the design of the complete system architecture, the relational database schema, the 3D visualization pipeline, and the core user-facing modules (home, car collection, car detail, manufacturer, country, parts, search, and comparison pages). Authentication is scoped to support user favourites; a full administrative CRUD interface is architecturally planned but treated as a future-phase deliverable. The seed dataset covers a representative sample of ten countries, twenty manufacturers, and forty car models with multiple variants, engines, transmissions, and over fifty individually documented parts — the schema is designed to scale to thousands of vehicles without structural change.

3.4 Key Features and Modules

Module	Description
Cinematic Homepage	Full-screen hero section with a rotating 3D vehicle, floating specification cards, and primary calls to action.
3D Car Viewer	React Three Fiber based viewer with orbit/zoom/pan controls, cinematic lighting rig, and preset camera views.
Exploded Component View	Animates a vehicle into its major assemblies (body, engine, transmission, suspension, brakes, wheels, interior, electronics) with a detail panel per

Module	Description
	component.
Car Detail Page	Dynamic route per vehicle presenting identity, performance, engine, transmission, dimensions, and fuel/energy data — fields adapt to the powertrain type.
Anatomy of the Machine	A dedicated parts encyclopedia covering exterior, engine, drivetrain, suspension, braking, wheels, interior, and electrical/EV components.
Country Explorer	Interactive country selection showing automotive history, major manufacturers, and associated models.
Manufacturer Pages	Brand profile (origin, founding year, headquarters, history) followed by a categorised model grid.
Variant System	Supports multiple variants per model (e.g., M3, M3 Competition, M3 Competition xDrive), each with independent specifications.
Advanced Search & Filters	Natural-language style search combined with structured filters for country, manufacturer, category, fuel, transmission, drivetrain, horsepower, top speed, price, and year.
Comparison Tool	Side-by-side comparison of 2–4 vehicles across performance, engine, transmission, dimensions, weight, fuel, and technology.
Engineering Mode	An alternate technical visualization layer showing component labels, airflow, power flow, and drivetrain layout.
Powertrain Visualizer	Animated diagram of energy flow for ICE, EV, and Hybrid drivetrains.
Car DNA Card	A visual attribute profile (performance, technology, luxury, track focus, comfort) derived from documented specification fields.
Admin-Ready Architecture	Route and data-layer structure prepared for a future administrative dashboard covering all core entities.

3.5 System Architecture

The application follows a three-tier architecture consisting of a presentation layer (Next.js/React client and server components), an application/service layer (reusable TypeScript query functions), and a persistence layer (Supabase / PostgreSQL with Row Level Security). The layers communicate as follows:

- **Presentation Layer** — Next.js pages and React components render the UI, including the 3D scene, and call typed data-access functions rather than querying the database directly.
- **Service / Data-Access Layer** — A dedicated set of functions (e.g., `getCars()`, `getCarBySlug()`, `getManufacturers()`, `getCountryBySlug()`, `getComparisonData()`) encapsulates all Supabase queries, keeping database logic out of UI components.
- **Persistence Layer** — Supabase provides the PostgreSQL database, object storage for images and 3D models, and Row Level Security policies that govern read/write access by role (public, authenticated, admin).

This separation ensures that the 3D rendering and animation logic, the UI component library, and the underlying data model can evolve independently — an important property given the scale the platform is intended to reach.

3.6 Database Design

The database is modelled as a normalized relational schema in PostgreSQL, mirroring the conceptual hierarchy Country → Manufacturer → Model → Variant → Specifications → Parts, as illustrated in the flowchart in Section 7. Core tables include:

Table	Purpose
countries	Country profile: name, slug, flag, description, automotive history.
manufacturers	Brand profile linked to a country: name, logo, founding year, headquarters.
car_models	A model line belonging to a manufacturer: name, generation, body type.
car_variants	A specific trim/variant of a model, with production years and price.
car_specifications	Links a variant to its engine, transmission and drivetrain; stores horsepower, torque, top speed, 0–100 km/h.
engines / transmissions / drivetrains	Reusable mechanical component definitions referenced by specifications.
dimensions / performance_specs / fuel_specs / ev_specs	Powertrain-specific detail tables, populated only where relevant.
car_parts / part_categories / car_part_relationships	The parts encyclopedia and its category taxonomy and cross-references.
car_images / car_3d_models	Media references (image URLs, GLB/GLTF model URLs) for each vehicle.
features / car_features / categories	Feature tagging and vehicle categorisation (hypercar, SUV, EV, etc.).
profiles / favorites	User profile and saved-vehicle data for authenticated users.

Foreign keys enforce referential integrity across the hierarchy, and indexes are defined on frequently filtered columns — country_id, manufacturer_id, category_id, model_id, variant_id, fuel_type, horsepower, and top_speed — to keep search and filter queries performant as the dataset grows.

3.7 3D Visualization Architecture

The 3D experience follows a consistent scene pipeline: Scene → Camera → Lights → Model → Controls → Animation → Interaction. A single shared React Three Fiber canvas hosts a scene graph in which a key light, rim light, fill light, and environment map produce a cinematic lighting setup. GLTF/GLB models are lazily loaded via Suspense so that only the currently visible vehicle consumes GPU and network resources. Camera behaviour is handled through damped orbit controls with scripted transitions between preset views (360°, front, rear, side, interior, engine, wheels, brakes), and the exploded-view mode drives per-part position animation using GSAP-timed transforms rather than physics simulation, keeping the interaction predictable and performant on mid-range devices.

3.8 Technology Stack Justification

Next.js and React were selected for their mature support of server components, dynamic routing, and image optimisation, all of which are required for an SEO-friendly, content-heavy site with thousands of potential vehicle pages. React Three Fiber was chosen over raw Three.js because it allows the 3D scene to be expressed declaratively alongside the rest of the React component tree, simplifying state synchronisation between UI controls and the 3D scene. Supabase was selected over a custom backend because it provides a managed PostgreSQL database, object storage, authentication, and Row Level Security out of the box,

which matches the project's requirement to avoid an unnecessary custom backend or containerised infrastructure (no Docker) while still supporting a fully relational data model.

3.9 Security Architecture

Supabase Row Level Security (RLS) governs data access at the database level rather than solely in application code. Public (unauthenticated) users are restricted to SELECT queries on car, manufacturer, and country data. Authenticated users may additionally insert their own favourites and update their own profile. Administrative write access (insert, update, delete across all entities) is reserved for an admin role. Service-role credentials are never exposed to the frontend; all client-side access uses the public anonymous key together with RLS policies, and secrets are managed exclusively through environment variables.

3.10 Advantages of the Proposed System

- Consolidates fragmented automotive information into one consistent, relationally structured platform.
- Provides an interactive, visual understanding of vehicle mechanics through 3D and exploded views rather than static text.
- Scales cleanly from a small seed dataset to thousands of vehicles without schema changes.
- Is brand-neutral, enabling fair, side-by-side comparison across manufacturers and countries.
- Is architected for future extension — admin CRUD, authentication, and additional data sources can be layered on without redesign.

3.11 Limitations and Future Scope

- The current seed dataset is representative rather than exhaustive; expanding coverage to every global manufacturer and model is an ongoing data-entry effort.
- 3D models are only as detailed as the GLB assets available; where a real model is unavailable, the system falls back to a placeholder representation.
- The administrative dashboard is architecturally prepared but not fully implemented in the current phase.
- Future work includes real-time pricing integration, user reviews, and multi-language support.

3.12 Application Route Map

The application exposes a semantic, SEO-friendly set of routes, each mapping to a corresponding page component and data-fetching function:

Route	Description
/	Cinematic homepage with hero 3D viewer and navigation entry points.
/cars	Global car collection grid with search and filters.
/cars/[manufacturer]/[model]/[variant]	Dynamic car detail page (e.g. /cars/porsche/911/turbo-s).
/manufacturers	Grid of all manufacturers.
/manufacturers/[slug]	Individual manufacturer profile and model grid (e.g. /manufacturers/porsche).
/countries	Interactive world map / country selection page.
/countries/[slug]	Country overview, history, and associated manufacturers (e.g. /countries/germany).

Route	Description
/parts	"Anatomy of the Machine" — parts encyclopedia landing page.
/parts/[slug]	Individual part detail page (e.g. /parts/turbocharger).
/compare	Side-by-side comparison tool for 2-4 vehicles.
/admin	Administrative dashboard (future-phase; architecture reserved).

4. SOFTWARE USED

The following software, tools, and platforms were used across design, development, database management, and deployment of the AURIX platform.

Category	Software / Tool	Purpose
Code Editor	Visual Studio Code	Primary IDE for writing and debugging the application code.
Runtime	Node.js	JavaScript/TypeScript runtime used to build and run the Next.js application.
Framework	Next.js (React)	Full-stack React framework used for routing, rendering, and API integration.
Package Manager	npm	Dependency installation and script management.
Database Platform	Supabase	Managed PostgreSQL database, authentication, and storage backend.
Database Engine	PostgreSQL	Relational database engine underlying Supabase.
Version Control	Git & GitHub	Source code versioning and collaborative development.
Design Tool	Figma	UI/UX layout, wireframing, and visual design planning.
3D Asset Tools	Blender / GLTF utilities	Preparation and optimisation of GLB/GLTF 3D vehicle models.
Browser Tools	Chrome DevTools	Performance profiling, responsive testing, and debugging.
Deployment Platform	Vercel	Hosting and continuous deployment of the Next.js application.
Documentation	Markdown / README	Project documentation and setup instructions.

4.1 Hardware and Software Requirements

The following minimum configuration is recommended for comfortable development and for end-users running the deployed application in a modern browser.

Requirement	Development Machine	End-User (Browser)
Processor	Intel i5 / AMD Ryzen 5 or higher	Any modern dual-core processor
RAM	8 GB minimum (16 GB recommended)	4 GB minimum
Graphics	Integrated GPU sufficient; dedicated GPU improves 3D preview performance	WebGL 2.0 compatible GPU/driver
Operating System	Windows 10/11, macOS, or Linux	Any OS with a modern browser
Browser	Latest Chrome / Edge / Firefox	Latest Chrome, Edge, Firefox, or Safari
Internet Connection	Required for Supabase and package installation	Required to load 3D assets and data

These figures represent recommended minimums; actual performance may vary depending on browser, device, and network conditions.

5. PROGRAMMING LANGUAGE USED

AURIX is built using a modern, strongly typed web technology stack. Each language plays a distinct role in the overall system:

Language / Technology	Role in the Project
TypeScript	Primary application language for both frontend components and backend data-access functions; provides static typing across the entire codebase to reduce runtime errors.
JavaScript (ES2020+)	Underlying runtime language executed by the browser and Node.js; TypeScript compiles down to modern JavaScript.
SQL (PostgreSQL dialect)	Used to define the relational schema, foreign keys, indexes, and Row Level Security policies within Supabase.
HTML5	Semantic document structure rendered by React components.
CSS3 / Tailwind CSS	Utility-first styling used to implement the dark-luxury visual design system, layout, and responsiveness.
GLSL (via Three.js/WebGL)	Shader logic underlying materials, lighting, and reflections in the 3D vehicle viewer.

This combination allows the same type definitions to describe data shapes on both the client and the data-access layer, reducing mismatches between what the database returns and what the UI expects.

6. APPLICATION AREA

While AURIX is presented here as an academic mini project, the underlying concept and architecture are applicable to a number of real-world domains:

- Automotive Education — helping engineering and design students visually understand vehicle construction, mechanical layout, and powertrain flow.
- Digital Automotive Museums — presenting the automotive history and notable models of different countries in an interactive format.
- Dealership and Showroom Kiosks — an interactive 3D configurator-style display for showcasing vehicle specifications to walk-in customers.
- Automotive Journalism and Research — a structured reference source for comparing specifications across manufacturers and markets.
- Enthusiast and Community Platforms — a shared, brand-neutral space for car enthusiasts to explore, compare, and discuss vehicles.
- Manufacturer Marketing Microsites — the 3D viewer and exploded-view architecture can be adapted into a single-brand configurator.
- E-Learning Platforms — engineering coursework on internal combustion, hybrid, and electric drivetrains can use the powertrain visualizer as a teaching aid.

7. FLOWCHART

The following flowcharts describe the data hierarchy used by the AURIX database and the end-to-end process flow experienced by a user navigating the application.

7.1 Data Hierarchy Flowchart

This flowchart illustrates how automotive data is structured relationally, from the broadest entity (Country) down to the most granular (individual Parts / Components).

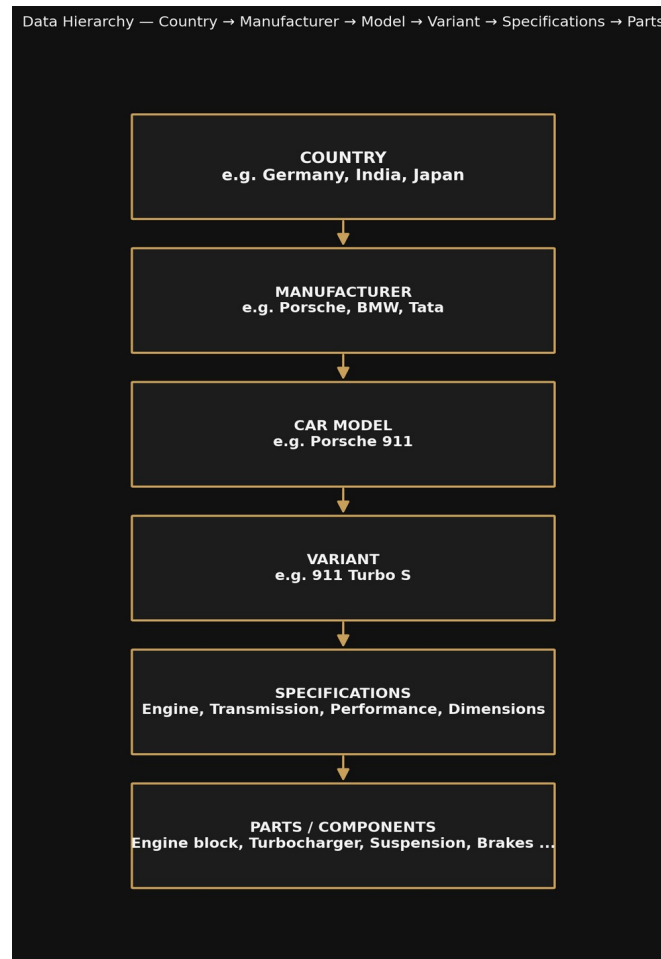


Figure 7.1 — Data Hierarchy: Country → Manufacturer → Model → Variant → Specifications → Parts

7.2 System Process Flowchart

This flowchart traces a typical user journey through the application, beginning at the loading screen and branching through browsing, search, vehicle exploration, comparison, and favouriting.

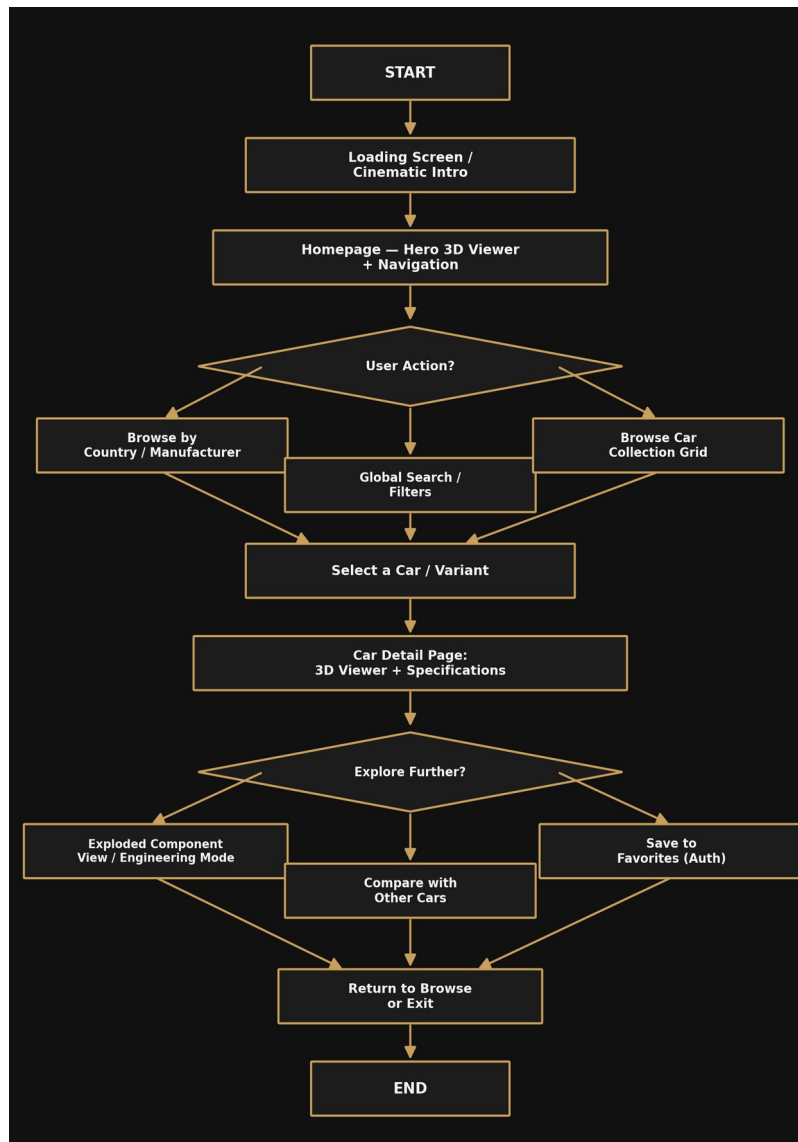


Figure 7.2 — End-to-End Application Process Flow

8. INPUT

AURIX accepts input from two distinct categories of users: general visitors interacting with the public-facing site, and administrators responsible for populating and maintaining the vehicle database. The table below summarises the primary inputs handled by the system.

Input Type	Description	Example
Global Search Query	Free-text query interpreted against manufacturer, model, category and specification data.	"German supercars above 300 km/h"
Filter Selections	Structured filter parameters applied to the car collection.	Country: Japan; Fuel: Electric; Transmission: Single-Speed EV
Navigation Selection	Selection of a country, manufacturer, model, or variant while browsing.	Germany → Porsche → 911 → 911 Turbo S
Comparison Selection	Selection of 2-4 vehicles to compare side by side.	BMW M3 Competition vs. Mercedes-AMG C63
3D Viewer Controls	Mouse/touch input for orbit, zoom, pan, and preset camera view buttons.	Drag to orbit; tap "ENGINE" view
Exploded View Trigger	Activation of "Explore Components" to separate a vehicle into assemblies.	Click "EXPLORE COMPONENTS"
Authentication Input	Sign-up / login credentials for saving favourites.	Email and password
Admin Data Entry (future phase)	Structured entry of new countries, manufacturers, models, variants, specifications, parts, images and 3D model files.	New variant: "M3 CS" with engine, transmission and pricing data

9. SCREENSHOT OF OUTPUT

The figures below present the intended interface of the AURIX platform as conceptual UI wireframes, derived directly from the design specification described in Section 3. They illustrate structure, layout, and information hierarchy rather than final pixel-level visuals. Once the application is fully implemented and deployed, these should be replaced with actual screenshots captured from the running website.

9.1 Homepage — Hero Section

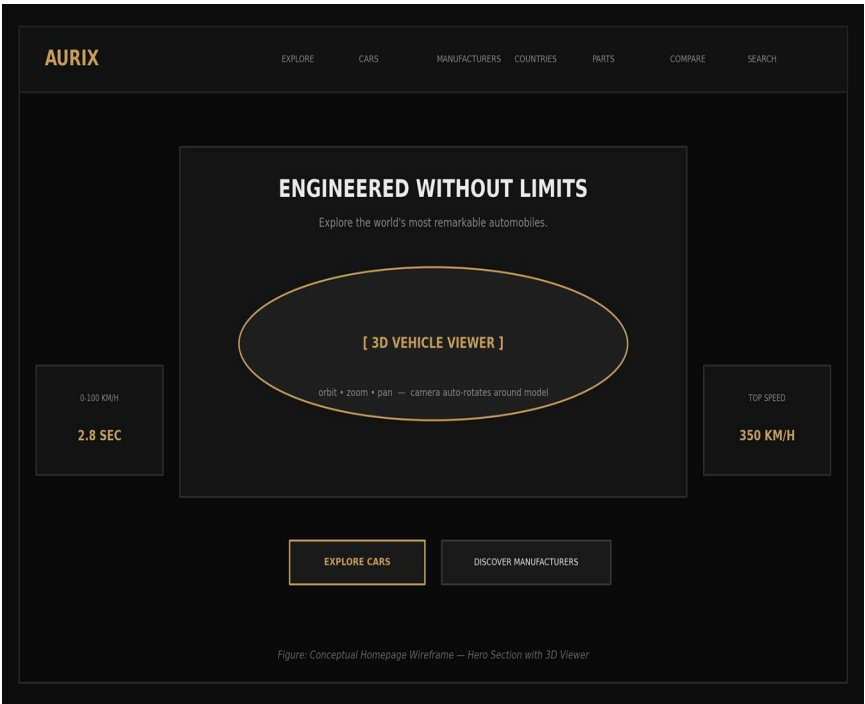


Figure 9.1 — Conceptual Homepage Wireframe (Hero Section with 3D Viewer)

9.2 Car Detail Page

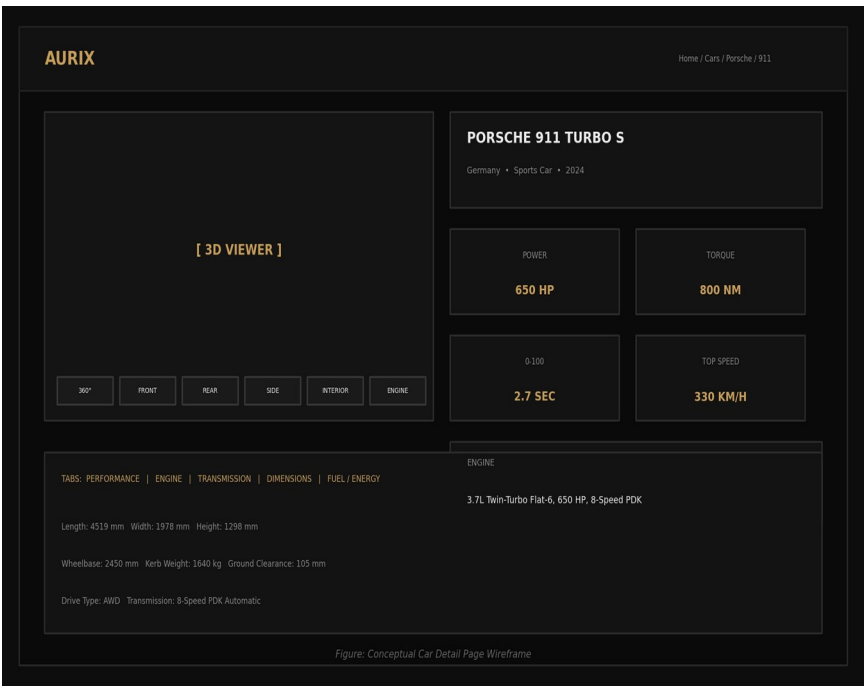


Figure 9.2 — Conceptual Car Detail Page Wireframe

9.3 Search Results / Car Collection Grid

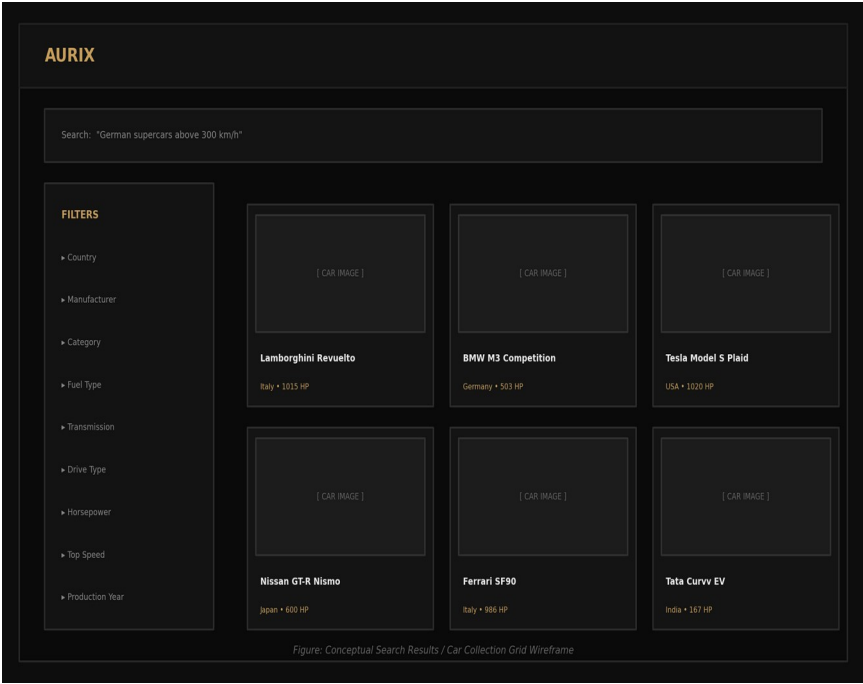


Figure 9.3 — Conceptual Search Results / Car Collection Grid Wireframe

Note: All vehicle names, figures, and images shown above are illustrative placeholders used to demonstrate the interface layout, and do not represent verified manufacturer data.

10. CONCLUSION

The AURIX — Global Automotive Intelligence project demonstrates how a modern web technology stack — Next.js, React, TypeScript, React Three Fiber, and Supabase — can be combined to build a single platform that unifies global automotive data, interactive 3D visualization, and structured engineering information under one consistent, relationally sound architecture.

Through the design work carried out in this project, a clear database schema, system architecture, 3D rendering pipeline, and complete route map were established, covering the full hierarchy from country and manufacturer data down to individual vehicle parts. The interface design emphasises a premium, minimal, dark-luxury visual language appropriate for an automotive technology product, while the technical architecture emphasises scalability, security (via Row Level Security), and performance (via lazy loading and optimised asset delivery).

Beyond its academic scope, the project illustrates practical skills in full-stack development, relational database design, 3D web graphics, responsive UI/UX design, and secure application architecture — skills directly transferable to real-world automotive, e-commerce, and data visualization platforms. With its modular, admin-ready structure, AURIX is well positioned for future expansion into a fully functional, continuously updated automotive intelligence platform.

REFERENCES

- Next.js Documentation — <https://nextjs.org/docs>
- React Three Fiber Documentation — <https://docs.pmnd.rs/react-three-fiber>
- Three.js Documentation — <https://threejs.org/docs>
- Supabase Documentation — <https://supabase.com/docs>
- PostgreSQL Documentation — <https://www.postgresql.org/docs>
- Tailwind CSS Documentation — <https://tailwindcss.com/docs>
- GSAP Documentation — <https://gsap.com/docs>
- MDN Web Docs (HTML, CSS, JavaScript) — <https://developer.mozilla.org>